Master of Science in Informatics at Grenoble
Master Informatique
Specialization Type your Option Here

Your Title

Your Name

Defense Date, 2023

Research project performed at YOUR LAB

Under the supervision of:
Your Supervisor

Defended before a jury composed of:
Head of the jury
Jury member 1
Jury member 2

Month 2023

Abstract

Your abstract goes here...

Acknowledgement

I would like to express my sincere gratitude to .. for his invaluable assistance and comments in reviewing this report... Good luck :)

Résumé

Your abstract in French goes here...

Contents

Abstract **i**

Acknowledgement **i**

Résumé **i**

0 Introduction **1**

0.1 Machine Learning 1

0.2 Kernel Methods 4

0.2.1 Translation Invariant Kernels 6

— 0 —
Introduction

Structure:

1. goal of the report
2. What is machine learning
3. supervised machine learning
4. kernel methods
5. main results in kernel methods theory

The goal of the present work is to further investigate the capabilities of kernel methods in the context of memory-constrained systems. We will explore the theoretical foundations of kernel methods, adapting previous work done in the Random Fourier Features framework, analyze their performance in various scenarios, and propose novel approaches to optimize their efficiency while maintaining high accuracy. In particular we will focus on the quantization of kernel methods, which is a promising avenue for reducing memory usage without significantly compromising performance. We apply this technique in an empirical loss minimization setting, exploiting mini-batches SGD algorithms, and we show that it can achieve competitive results compared to unquantized methods, while significantly reducing memory requirements. We start with a brief introduction to machine learning, supervised learning, and kernel methods, before delving into the main results of our work.

0.1 Overview of Machine Learning

Machine learning is a subfield of artificial intelligence that focuses on the development of algorithms and statistical models that enable computers to perform tasks without previous coded instructions. The core idea behind machine learning is to allow systems to learn from data, identify patterns, and make decisions with minimal human intervention

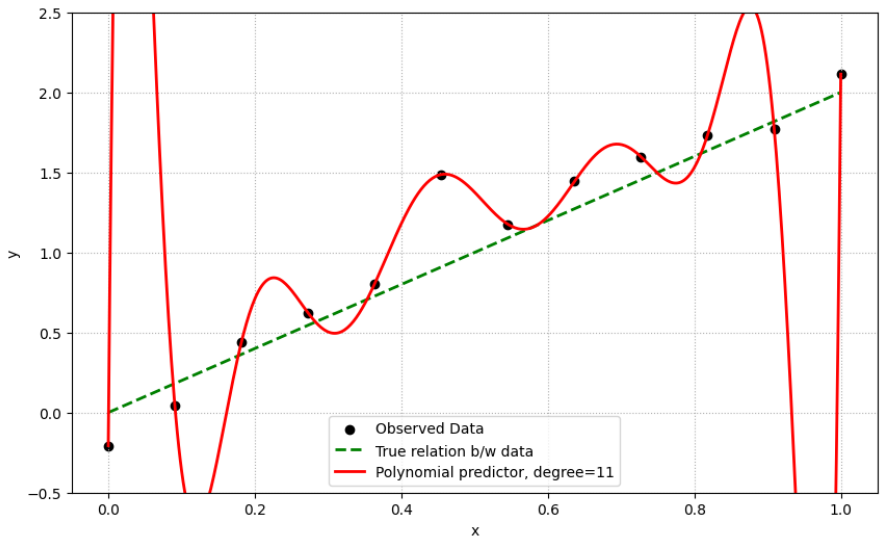

Figure 0.1: Example of overfitting: the model fits the training data perfectly but fails to generalize to new data, which are distrubuted as: $y = 2x + \varepsilon$, where ε is a Normal noise term.

on data not previously seen. Machine learning algorithms can be broadly categorized into three main types: supervised learning, unsupervised learning, and reinforcement learning. We are going to focus on supervised learning, which is the most widely used type of machine learning, where the algorithm is trained on a labeled dataset, meaning that each training example is paired with an output label.

Assume we have a dataset of n examples, where each example consists of an input vector $x_i \in \mathbb{R}^d$ and a corresponding output label $y_i \in \mathbb{R}$. The goal of supervised learning is to learn a function $f : \mathbb{R}^d \to \mathbb{R}$ that can predict the output label y for new, unseen input vectors x. To do so, we "train" the model by minimizing a loss function that measures the discrepancy between the predicted outputs and the true labels in the training data, i.e. we solve the following optimization problem:

$$\min_{f \in \mathscr{H}} \frac{1}{n} \sum_{i=1}^{n} L(f(x_i), y_i) + \lambda R(f) \tag{1}$$

where $\mathscr{H}$ is a suitable space of functions (predictors), L is a loss function that quantifies the error of the predictions, R is a regularization term, i.e. it prevents f from varing too sharply in order to fit the knonwn data hence only "memorizing" it, and λ is a regularization parameter that controls the trade-off between fitting the training data and keeping the model simple. One of the most common choiches for the function L is the mean squared error, defined as $L(f(x_i), y_i) = (f(x_i) - y_i)^2$, which is widely used for regression problems and the one we will be using in this work. A common choiche for the regularization term R is the squared norm of f in a suitable function space, which is

known as Tikhonov regularization and is widely used in kernel methods.

As previously mentioned, choosing the right function space $\mathscr{H}$ is crucial for the success of the learning algorithm. If the space is too large, in a sense that we will further clarify in the next sections, the model may overfit the training data, meaning that it will perform well on the training data but poorly on new, unseen data. On the other hand, if the space is too small, the model may underfit the training data, meaning that it will perform poorly on both the training data and new, unseen data. It is also important to work with a function space that allows for efficient optimization, as the training process involves solving an optimization problem, which could be unfeasible if the function space is too complex. In the example given in Figure 0.1 we allow the function space to include higly interpolating polynomials, which leads to a zero training error but once evaluated on new data, the model fails to generalize and performs poorly, which is a clear example of overfitting.

We give now more formal definitions.

Definition 0.1.1 *Let $\mathscr{X} \subseteq \mathbb{R}^d$ be the input space and $\mathscr{Y} \subseteq \mathbb{R}$ the output space. Over the product space $\mathscr{X} \times \mathscr{Y}$ we assume there is a probability distribution function ρ that generates the data, and we denote by ρ_x the marginal distribution of ρ on $\mathscr{X}$ and with $\rho(y|x)$ the conditional distribution of ρ given $\mathscr{X}$.*

As previously mentioned, the goal of supervised learning is to learn a function $f : \mathscr{X} \to \mathscr{Y}$ that can predict the output label y for new, unseen input vectors x; so the next definition comes naturally:

Definition 0.1.2 *Let $\mathscr{H}$ a space of function from $\mathscr{X}$ to $\mathscr{Y}$, each of the function f in $\mathscr{H}$ is called a predictor.*

Now we define the quantity with respect to which we choose the best predictor:

Definition 0.1.3 *A function $l : \mathscr{Y} \times \mathscr{Y} \to \mathbb{R}_{\geq 0}$ is called loss function. The most common loss function is the mean squared error, defined as:*

$$l(y', y) := ||y' - y||^2$$

Definition 0.1.4 *We call expected risk the function:*

$$R(f) := \mathbb{E}_\rho(l(f(x), y)) = \int_{\mathscr{X} \times \mathscr{Y}} l(f(x), y) d\rho(x, y)$$

where $f \in \mathscr{H}$ suitable function space.

The best predictor in $\mathscr{H}$ is defined as the minimizer of the expected risk function. Theoretically the best possible predictor is the following:

Definition 0.1.5 *We call Bayes Predictor the function:*

$$f^\star(x) = \underset{z\in\mathscr{Y}}{\operatorname{argmin}}\, \mathbb{E}_{y\sim\rho(y|x)}(l(y,z))$$

and the corresponding risk is called Bayesian Risk, which is the best theoreticaly achievable risk[1]*:*

$$R(f^\star) = \mathbb{E}_{x\sim\rho_X}\left(\inf_z \mathbb{E}_{y\sim\rho(y|x)}(l(z,y))\right)$$

It is clear that we do not have access to ρ, otherwise we could (numericaly) compute the function $f^\star$ and get a state-of-the-art predictor. Since we do not have ρ, we cannot even calculate $R(\cdot)$; we have then to rely on the following:

Definition 0.1.6 *Let $(x_i, y_i)_{i=1,\dots,n} \in \mathscr{X} \times \mathscr{Y}$ be the known dataset, let f an estimator. We define the Empirical Risk as:*

$$\hat{R}(f) = \frac{1}{n}\sum_{i=1}^{n} l(f(x_i), y)$$

The minimization of the empirical risk is a core idea behind the majority of machine learning tasks.

When learning the best regressor, with respect to the choosen loss, an ideal situation would be it to be a linear function, hence easier to learn in computational framework. Unfortunatly this is not always the case, but there are tecniques to non-linear regression that allow us to tackle this issue. The main idea is to map the input space to a higher dimensional space where the data is more likely to be linearly separable, and then learn a linear regressor in this higher dimensional space. The problem of choosing the right function space is know as model selection, and it is a crucial aspect of supervised learning.

0.2 Kernel Methods

We star defining a kernel function:

Definition 0.2.1 *Let $\mathscr{X} \subseteq \mathbb{R}^d$ be the input space, we call kernel function, a simmetric function $k : \mathscr{X} \times \mathscr{X} \to \mathbb{R}$ s.t. for all $n \in \mathbb{N}, x_1, \dots, x_n \in \mathscr{X}$ and $c_1, \dots, c_n \in \mathbb{R}$, we have:*

$$\sum_{i=1}^{n}\sum_{j=1}^{n} c_i c_j k(x_i, x_j) \geq 0$$

This last condition is called positive semi-definiteness and it ensures that the matrix of the kernel values, called Gram matrix, is positive semi-definite. It is related to the definition of a kernel function through the following theorem:

[1] This can be easily seen, since $d\rho_X$ is a positive measure.

Theorem 0.2.2 *Let $\mathscr{X} \subseteq \mathbb{R}^d$ be the input space and $k : \mathscr{X} \times \mathscr{X} \to \mathbb{R}$ a simmetric function. Then k is a kernel function if and only if there exists a Hilbert space $\mathscr{H}$ and a feature map $\phi : \mathscr{X} \to \mathscr{H}$ such that for all $x, y \in \mathscr{X}$:*

$$k(x,y) = \langle \phi(x), \phi(y) \rangle_{\mathscr{H}}$$

where $\langle \cdot, \cdot \rangle_{\mathscr{H}}$ denotes the inner product in $\mathscr{H}$. In this case $\mathscr{H}$ is called Reproducing Kernel Hilbert Space, and the map ϕ is called feature map.

The theorem, due to Aronszajn in the 1950s, allows us to implicitly map the data to a higher dimensional space where it is more likely to be linearly separable, furthermore, as we are about to see, we do not need to know the explicit feature map, since most of the algorithms rely only on the "interaction between data", i.e. the value of the kernel function.

To see the link with learning theory we need some assumption. First we assume our predictors to be linear with respect to the parameters, i.e. of the form:

$$f_\theta : \mathscr{X} \to \mathbb{R}, \quad x \to \langle \theta, \phi(x) \rangle_{\mathscr{H}}$$

where $\theta \in \mathscr{H}$ is the parameter vector. Note that the function is not linear on the data x itself, but on their representation in the feature space $\mathscr{H}$. Furthermore we assume the loss function to be of the form:

$$L(f_\theta, (x_i, y_i)_{i=1,\dots,n}) = \frac{1}{n} \sum_{i=1}^{n} l(f_\theta(x_i), y_i) + \lambda ||\theta||^2$$

where l is a loss function and $\lambda \geq 0$ is the regularization parameter. Now we can state the following theorem, which allows us to restrict the optimal parameters space.

Theorem 0.2.3 *Consider a feature map $\phi : \mathscr{X} \to \mathscr{H}$. Let $(x_1, \dots, x_n) \in \mathscr{X}^n$, and assume that the functional $\Psi : \mathbb{R}^{n+1} \to \mathbb{R}$ is strictly increasing with respect to the last variable. Then the infimum of*

$$\Psi\left(\theta, \phi(x_1), \dots, \phi(x_n), ||\theta||^2\right)$$

can be obtained by restricting to a vector θ in the span of $\phi(x_1), \dots, \phi(x_n)$; that is, of the form

$$\theta = \sum_{i=1}^{n} \alpha_i \phi(x_i)$$

with $\alpha \in \mathbb{R}^n$.

It comes straightforward that the loss function we defined above satisfies the hypothesis of the theorem, hence we can restrict our search space of optimal parameters to the space spanned by the feature map of the training data.

0.2.1 Translation Invariant Kernels

We now focus on a translation invariant kernels, which will be the starting point of this work.